

MoviCare

System Design & Architecture

1. System Architecture Overview

MoviCare follows a client-server architecture pattern with React Native mobile clients communicating with Firebase backend services. The architecture emphasizes real-time data synchronization, offline capabilities, and scalable cloud infrastructure.

1.1 High-Level Architecture

- **Presentation Layer:** React Native components with Expo Router navigation
- **Business Logic Layer:** React Context API for state management, custom hooks for data operations
- **Data Layer:** Firebase Firestore for persistence, Firebase Authentication for security
- **External Services:** Expo Location, Firebase Cloud Messaging, Expo Notifications

2. Frontend Architecture

2.1 Navigation Structure

Expo Router provides file-based routing with support for role-based navigation flows. The navigation hierarchy separates Senior, Family, and Caregiver experiences while maintaining code reusability.

```
app/ ├── (auth)/ | ├── login.tsx | └── register.tsx ├── (senior)/ | ├── home.tsx | ├── profile.tsx | ├── health.tsx | ├── medications.tsx | ├── (family)/ | ├── dashboard.tsx | ├── seniors/[uid].tsx | └── notifications.tsx └── (caregiver)/ ├── dashboard.tsx └── schedule.tsx
```

2.2 State Management

Application uses React Context API with custom hooks pattern for clean separation of concerns:

- **AuthContext:** User authentication state, role management, profile data
- **NotificationContext:** Real-time notification handling and badge counts
- **HealthContext:** Health metrics caching and synchronization

2.3 Component Architecture

Components follow atomic design principles with emphasis on reusability and accessibility:

- **Atoms:** Button, Input, Text, Icon - fundamental UI building blocks
- **Molecules:** FormField, HealthMetricCard, NotificationBadge - composite components
- **Organisms:** MedicationList, HealthDashboard, EmergencyButton - complex UI sections
- **Templates:** DashboardLayout, ProfileLayout - page-level structures

3. Backend Architecture

3.1 Database Design

Firestore collections are structured for efficient querying and real-time updates:

```
users/ {userId}/ - email, role, displayName, createdAt - profile: { dateOfBirth, address, phone, emergencyContact } - healthMetrics: { conditions[], allergies[], bloodPressure[], bmi[] } relationships/{relationshipId}/ - seniorId, familyMemberId/caregiverId - type: 'family' | 'caregiver' - permissions: { viewHealth, viewLocation, receiveAlerts } medications/ {medicationId}/ - seniorId, name, dosage, frequency, schedule - active, createdAt, createdBy appointments/ {appointmentId}/ - seniorId, dateTime, location, purpose - attendees[], reminders[] notifications/ {notificationId}/ - recipientId, recipientRole, type, message - metadata: { location, healthData, severity } - read, timestamp
```

3.2 Security Rules

Firestore security rules enforce role-based access control:

- Seniors can read/write own profile and health data - Family members can read linked senior data based on relationship permissions - Caregivers have read-only access to assigned senior data - All users can only create notifications for authorized recipients - Location data is only accessible during active check-ins or emergencies

3.3 Real-Time Synchronization

Firestore listeners enable real-time updates across all connected clients:

- Health metrics updates propagate immediately to family dashboards
- Emergency alerts trigger instant notifications to all authorized recipients
- Medication schedules sync across senior and caregiver views
- Check-in status updates reflect in real-time on family member devices

4. Key Features Implementation

4.1 Authentication Flow

Firebase Authentication with email/password provides secure user management:

1. User registers with email, password, and role selection
2. Firebase creates auth account and returns user ID
3. Application creates Firestore user document with profile data
4. User is redirected to role-appropriate home screen
5. Auth state persists across app sessions

4.2 Emergency Alert System

One-touch emergency alerts combine location services and push notifications:

1. Senior presses emergency button (long-press on check-in button)
2. Expo Location captures current GPS coordinates
3. System queries relationships to identify all family members and caregivers
4. Notification documents created for each recipient with location metadata
5. Firebase Cloud Messaging sends immediate push notifications

6. Recipients can view senior location on map and initiate contact

4.3 Health Metrics Tracking

Health data entry includes validation and automatic calculations:

- Blood pressure: Systolic (90-200) and diastolic (60-130) with timestamp
- BMI: Auto-calculated from height/weight inputs, categorized by WHO standards
- Conditions & Allergies: Managed as arrays with add/remove operations
- Historical data: Stored as timestamped arrays for trend analysis

4.4 Notification System

Targeted notification delivery prevents information overload:

- Each notification includes recipientRole and recipientUid fields
- Firestore queries filter notifications by current user's ID and role
- Medication reminders target only the senior
- Check-in confirmations notify family members, not the senior
- Emergency alerts go to all authorized recipients

5. Data Flow Diagrams

5.1 Check-In Flow

1. Senior taps check-in button
2. App requests location permission (if not granted)
3. Expo Location retrieves current coordinates
4. Notification created with senior's status and location
5. Family members receive push notification
6. Family dashboard updates with check-in marker and timestamp

5.2 Medication Reminder Flow

1. Caregiver or senior creates medication schedule
2. Expo Notifications schedules local notifications based on timing
3. At scheduled time, notification triggers on senior's device

4. Senior confirms medication taken (optional)
5. Confirmation updates Firestore, syncs to caregiver view
6. Missed medications generate alerts to caregivers after grace period

6. Performance Optimization

6.1 Data Caching

- Firestore persistence enabled for offline access
- User profile cached in AsyncStorage
- Health metrics cached locally with background sync
- Images optimized and cached using Expo Image

6.2 Query Optimization

- Firestore queries use compound indexes for efficiency
- Pagination implemented for large data sets (notifications, history)
- Real-time listeners scoped to minimum necessary data
- Denormalized data for frequently accessed information

7. Accessibility Considerations

- All interactive elements have minimum 44x44pt touch targets
- Screen reader support with semantic labels
- High contrast mode with 4.5:1 minimum ratio
- Text scalable to 200% without breaking layout
- Critical actions require confirmation dialogs
- Error messages are clear and actionable

8. Testing Strategy

8.1 Unit Testing

- Custom hooks tested with React Testing Library

- Business logic functions tested with Jest
- Data validation functions verified with edge cases

8.2 Integration Testing

- Authentication flows tested end-to-end
- Firestore operations verified with emulator
- Notification delivery tested across roles

8.3 User Acceptance Testing

- Usability testing with senior focus group
- Accessibility audit with assistive technology users
- Family member feedback on dashboard clarity

9. Deployment & DevOps

9.1 Build Process

- Expo EAS Build for production iOS and Android builds
- Environment variables managed via .env files
- Automated testing in CI/CD pipeline

9.2 Monitoring

- Firebase Analytics for user behavior tracking
- Crashlytics for error monitoring and reporting
- Performance monitoring for app vitals

10. Future Technical Enhancements

- Offline-first architecture with sync conflict resolution
- GraphQL API layer for more efficient data fetching
- WebRTC integration for video calling capabilities
- Machine learning models for health anomaly detection

- HealthKit/Google Fit integration for automated data collection